

claude-windows / df5d5fb2-534a-447a-8181-b222f94655f3

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\df5d5fb2-534a-447a-8181-b222f94655f3\subagents\workflows\wf\_133acdee-dab\agent-ac8f9c159e4b544a3.jsonl`
- SHA-256: `13b6700c7ea0b70d8fbbb943125a79fc2898930d6d38c18bb4df55327a3d56aa`
- Source modified: `2026-07-06T17:29:52+00:00`
- Imported at: `2026-07-08T15:59:35+00:00`
- project: `wf\_133acdee-dab`
- session_id: `df5d5fb2-534a-447a-8181-b222f94655f3`

Transcript

1. user (2026-07-06T17:29:03.349Z)

You are an ADVERSARIAL VERIFIER. Try HARD to REFUTE the following review finding by reading the ACTUAL code in C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer (and the diff at C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-indexer/df5d5fb2-534a-447a-8181-b222f94655f3/scratchpad/wf294_fix.diff). Default to REFUTED if you cannot concretely reproduce the defect by pointing at real code lines. A finding is CONFIRMED only if you can trace a concrete input/state that produces the claimed wrong behavior in the code AS WRITTEN (not as you imagine it). If the claimed line/behavior does not exist, or a guard/caller already handles it, or it is out-of-scope speculation, mark REFUTED. If genuinely unclear after reading, mark UNCERTAIN.

FINDING (from the completeness lens):

severity=high | backend/pipeline/segmenters/yolo_hybrid.py:471

summary: yolo_hybrid ? the DEFAULT segmenter and a parallel worker-dispatched hybrid ? was left entirely untouched and still returns bare [] on every failure path, so it reproduces the wf294 silent-failure shape while the trajectory twin was fixed, creating a yolo-vs-wasb/tracknet inconsistency.

detail: yolo_hybrid.process_video imports no Failure and returns bare [] on: missing API key / unknown sport / unknown provider (via _resolve_provider returning None ? line 470-471), and invalid video_duration<=0 (line 482). Its return annotation is still List[Dict[str,Any]] (line 465) with no Failure branch anywhere. It is registered (line 560), worker-dispatchable through the same cmd_infer path, and is in fact the configured default_segmenter (backend/config/models.py:181 default_segmenter='yolo_hybrid'). The wf294 fix touched only the trajectory base class; because cmd_infer's isinstance(result, Failure) branch is generic, yolo_hybrid can never trigger it. This is precisely the CODE_MAP 'twin-parity' footgun the task flagged: a same-class silent-failure hole remains in a sibling segmenter, so a default-segmenter run with a missing key still emits status="success"/0-segments.

failure_scenario: A cloud (or local) job uses the default segmenter yolo_hybrid with GEMINI_API_KEY unset. _resolve_provider logs an error and returns None; process_video returns bare [] at line 471. cmd_infer records status="success", pushes report.json status="success"/segment_count=0 and a done-marker rc=0, and the control plane (cached_process_result / _resume_one_remote_job) treats it as a completed empty match ? the identical wf294 silent failure, in the default code path, left unaddressed.

suggested_fix: Apply the same Failure contract to yolo_hybrid: have _resolve_provider return a Failure (message + is_recoverable) instead of None for missing-key/unknown-sport/unknown-provider, return a Failure on video_duration<=0, widen the annotation to Union[List, Failure], and add a test mirroring

tests/test_worker.py::test_cmd_infer_detector_failure_marks_report_and_marker_failed for yolo_hybrid. Otherwise the default segmenter silently regresses the exact incident the change was meant to fix.

Verify against the real code and return your verdict.

2. assistant (2026-07-06T17:29:05.208Z)

I'll verify this finding by reading the actual code.

3. user (2026-07-06T17:29:07.229Z)

```
440             # may yield multiple AI segments (one-to-many); confirmed_segment_id
keeps
441             # the last one ? the link table records "this candidate was
confirmed".
442             candidate_id = segment.get("_candidate_id")
443             if candidate_id is not None:
444                 self.db.link_candidate_to_segment(candidate_id, segment_id)
445             segments_data.append(
446                 {
447                     "id": segment_id,
448                     "start_time": start,
449                     "end_time": end,
450                     "duration": dur,
451                     "metadata": metadata,
452                 }
453             )
454         except Exception as e:
455             logger.warning(f"Skipping invalid segment struct: {e}")
456
457     return segments_data
458
459 def process_video(
460     self,
461     video_id: str,
462     video_path: str,
463     player_pool: List[str] = None,
464     max_frames: int = None,
465 ) -> List[Dict[str, Any]]:
466     self.person.lazy_load_model()
467
468     # Phase 0: resolve sport profile + AI provider (early-returns [] on any
failure).
469     provider = self._resolve_provider(video_path)
470     if provider is None:
471         return []
472
473     # Re-read the config values needed downstream (pure deterministic reads).
474     idx_cfg = self.config.indexing
475     sport_profile = sport_registry.get(self.config.sport)
476     provider_name = self.config.ai_provider
477     model_name = self.config.ai_model
478
479     video_duration = get_video_duration(video_path)
480     if video_duration <= 0:
481         logger.error("Invalid video duration.")
482         return []
483
484     chunk_duration = idx_cfg.chunk_duration
485     chunk_overlap = idx_cfg.chunk_overlap
486     velocity_thresh = (
487         idx_cfg.yolo_velocity_threshold
488     ) # Fraction of frame width per second
489     merge_gap = idx_cfg.dead_time_merge_gap
490     frame_skip = idx_cfg.yolo_frame_skip
491     tracker_config = idx_cfg.yolo_tracker
492     handoff_padding = idx_cfg.ai_handoff_padding
493     ai_max_workers = idx_cfg.ai_max_workers
494     log_candidates = idx_cfg.log_yolo_candidates
495     store_candidates = idx_cfg.store_yolo_candidates
496
497     # Snapshot of the params that produced these detections ? stored alongside each
```

```

498         # candidate so a fine-tuning run can reproduce / correlate against them.
499         detection_params = {
500             "velocity_thresh": velocity_thresh,
501             "merge_gap": merge_gap,
502             "frame_skip": frame_skip,
503             "tracker": tracker_config,
504             "detector_model": idx_cfg.detector_model,
505             "detector_score_threshold": idx_cfg.detector_score_threshold,
506             "min_action_window_duration": idx_cfg.min_action_window_duration,
507         }
508
509         chunks_dir = os.path.join(output_base(self.config), "chunks_yolo")
510         os.makedirs(chunks_dir, exist_ok=True)
511
512         # Phase 2: chunk ? per-chunk action windows ? cross-chunk overlap merge.
513         all_candidate_windows = self._collect_candidate_windows(
514             video_id,
515             video_path,
516             video_duration,
517             idx_cfg,
518             chunk_duration,
519             chunk_overlap,
520             velocity_thresh,
521             merge_gap,
522             frame_skip,
523             chunks_dir,
524         )
525
526         # Phase 2b: log + persist raw candidates (window index -> candidate_id).
527         candidate_ids = self._persist_candidates(
528             video_id,
529             all_candidate_windows,
530             detection_params,
531             log_candidates,
532             store_candidates,
533         )
534
535         try:
536             os.rmdir(chunks_dir)
537         except Exception:
538             pass
539
540         # Phase 3: AI Provider Handoff
541         gemini_segments = self._run_ai_handoff(
542             video_id,
543             video_path,
544             video_duration,
545             all_candidate_windows,
546             candidate_ids,
547             provider,
548             sport_profile,
549             provider_name,
550             chunk_duration,
551             handoff_padding,
552             ai_max_workers,
553         )
554
555         # Phase 4: DB Population
556         return self._populate_db(video_id, gemini_segments, model_name)
557
558
559         # Register the segmenter

```

```

1      diff --git a/backend/pipeline/segmenters/trajectory_hybrid.py
b/backend/pipeline/segmenters/trajectory_hybrid.py
2      index 503f6d8..f96b495 100644
3      --- a/backend/pipeline/segmenters/trajectory_hybrid.py
4      +++ b/backend/pipeline/segmenters/trajectory_hybrid.py
5      @@ -22,7 +22,7 @@ import concurrent.futures
6          import logging
7          import os
8          import time
9      -from typing import Any, Dict, List, Optional, Tuple
10     +from typing import Any, Dict, List, Optional, Tuple, Union
11
12     import cv2
13
14     @@ -30,6 +30,9 @@ from backend.config.models import IndexingConfig
15     from backend.pipeline.detectors.base import DetectorRunner
16     from backend.pipeline.segmenters.base import BasePlaySegmenter
17     from backend.providers import provider_registry
18     +
19     +# Standardized Success|Failure contract: a detector Failure is NOT a 0-rally result.
20     +from backend.results import Failure
21     from backend.sports import sport_registry
22     from backend.utils.paths import output_base
23     from backend.utils.video import extract_video_chunk, get_video_duration
24     @@ -174,10 +177,13 @@ class TrajectoryHybridSegmenter(BasePlaySegmenter):
25         video_path: str,
26         player_pool: Optional[List[str]] = None,
27         max_frames: Optional[int] = None,
28     - ) -> List[Dict[str, Any]]:
29     -     # override-ignore: the ABC declares the Result contract (Success|Failure)
30     -     # but every live segmenter still returns a bare list ? the documented
31     -     # deliberate-incremental gap (CODE_MAP gotchas; main.py handles both).
32     + ) -> "Union[List[Dict[str, Any]], Failure]":
33     +     # Partial adoption of the Result contract (the ABC declares Success|Failure).
34     +     # A DETECTOR failure ? the env healthcheck not ready, or run_predict timing out
35     +     # aborting with no trajectory ? returns a ``Failure`` so it is NOT confused
36     +     # genuine 0-rally video (which still returns a bare ``[]``). Every caller
37     +     # (worker.cmd_infer, main.py, orchestration) branches on ``isinstance(result,
38     +     # Failure)``;
39     +     # the bare-list success/empty paths are the documented incremental gap
40     +     # (CODE_MAP gotchas).
41     +     label = self.display_label
42     +     # --- Sport profile + AI provider wiring (identical across segmenters) ---
43     +     sport_name = self.config.sport
44     @@ -267,7 +273,12 @@ class TrajectoryHybridSegmenter(BasePlaySegmenter):
45         ok, msg = runner.healthcheck()
46         if not ok:
47             logger.error("%s detector environment not ready: %s", label, msg)
48             return []
49         +     # A detector FAILURE, not a 0-rally video ? surface it so the caller marks
50         +     # failed/retryable instead of writing an empty-but-"successful" result.
51         +     return Failure(
52         +         message=f"{label} detector environment not ready: {msg}",
53         +         is_recoverable=True,
54         +     )
55         logger.info("%s detector env OK: %s", label, msg)
56
57         # --- Phase 2: trajectory -> candidate rally windows ---
58     @@ -278,7 +289,17 @@ class TrajectoryHybridSegmenter(BasePlaySegmenter):
59         csv_path = runner.run_predict(video_path, out_dir, video_id=video_id)
60         if not csv_path:
61             logger.error("%s produced no trajectory; aborting.", label)

```

```

60      -          return []
61      +          # The cold-start / timeout failure mode (e.g. WASB native inference timed
out after
62      +          # timeout_sec, or the detector env yielded nothing). This is a DETECTOR
failure ? NOT
63      +          # a legitimately empty match ? so return a Failure the worker marks as
failed/retryable
64      +          # rather than a silent 0-segment "success" (the wf294 incident).
65      +          return Failure(
66      +              message=(
67      +                  f"{label} detector produced no trajectory (timed out or aborted) ?
"
68      +                  "detector failure, not a 0-rally video."
69      +              ),
70      +              is_recoverable=True,
71      +          )
72
73      points = runner.parse_trajectory_csv(csv_path)
74      logger.info(
75      diff --git a/backend/worker/__main__.py b/backend/worker/__main__.py
76      index 461d7ca..88c50f6 100644
77      --- a/backend/worker/__main__.py
78      +++ b/backend/worker/__main__.py
79      @@ -114,6 +114,42 @@ def _write_report_json(
80          )
81
82
83      +def _serialize_and_push_outputs(
84      +    scratch: str,
85      +    out_uri: str,
86      +    video_id: str,
87      +    segments: List[dict],
88      +    status: str,
89      +    storage_cfg: dict,
90      +    video_uri: str = "",
91      +) -> List[str]:
92      +    """Write intervals.csv + report.json under ``<scratch>/outputs/<video_id>/`` and
push both to
93      +    ``<out_uri>/outputs/<video_id>/``; return the pushed URIs.
94      +
95      +    Shared by the SUCCESS path and the failure path (``_fail``) so a detector
timeout/abort leaves a
96      +    ``status="failed"`` report.json in the bucket ? the control plane's source of truth
97      +    (``orchestration.cached_process_result`` / ``probe_process_cache`` treat any
non-``success``
98      +    report as "no usable result" ? retry, and the restart-recovery poll waits for
report.json to
99      +    EXIST) ? instead of a missing artifact or a false 0-segment ``"success"``. """
100     +    out_local = os.path.join(scratch, "outputs", video_id)
101     +    os.makedirs(out_local, exist_ok=True)
102     +    _write_intervals_csv(segments, os.path.join(out_local, "intervals.csv"))
103     +    _write_report_json(
104     +        video_id,
105     +        segments,
106     +        status,
107     +        os.path.join(out_local, "report.json"),
108     +        video_uri=video_uri,
109     +    )
110     +    out_prefix = out_uri.rstrip("/")
111     +    pushed: List[str] = []
112     +    for fn in sorted(os.listdir(out_local)):
113     +        uri = f"{out_prefix}/outputs/{video_id}/{fn}"
114     +        storage.put(os.path.join(out_local, fn), uri, config=storage_cfg)
115     +        pushed.append(uri)
116     +    return pushed

```

```

117 +
118 +
119 def _run_startup_checks(config: Any) -> bool:
120     """M5 per-profile startup checks for the worker (the compute process, so it probes
CUDA
121         best-effort). Logs warnings; returns False on a FATAL coherence error so the caller
aborts
122     @@ -237,9 +273,26 @@ def cmd_infer(args: argparse.Namespace) -> int:
123         )
124
125     def _fail(rc: int) -> int:
126         - # M6: on a worker-side FAILURE, write the done-marker (the REAL rc, 0 segments)
so a remote
127         - # dispatcher's bucket-poll terminates FAST + accurately instead of hanging
until its poll
128         - # budget expires. DEFAULT-OFF: no --done-uri ? no marker written (the unchanged
local run).
129         + # A worker-side FAILURE (validation, unknown segmenter, or a detector
timeout/abort the
130         + # segmenter surfaced as a Failure) must NOT masquerade as a 0-rally "success".
Write + push
131         + # a status="failed" report.json/intervals.csv (0 segments) so the control
plane's bucket
132         + # truth (submit-cache probe, restart-recovery) and the alpha-user result
surface see an
133         + # explicit FAILURE ? the wf294 silent-failure fix. Best-effort: a push hiccup
must never
134         + # mask the original failure. THEN the M6 done-marker (real rc + status) so a
remote
135         + # dispatcher's bucket-poll terminates FAST instead of hanging until its budget
expires.
136         + try:
137         +     _serialize_and_push_outputs(
138         +         scratch,
139         +         args.out_uri,
140         +         video_id,
141         +         [],
142         +         "failed",
143         +         storage_cfg,
144         +         str(getattr(args, "video_uri", "") or ""),
145         +     )
146         + except Exception as e: # never mask the real failure on an artifact-push
hiccup
147         +     logger.warning("[worker] could not push failure artifacts: %s", e)
148         + # DEFAULT-OFF: no --done-uri ? no marker written (the unchanged local run).
149         + if getattr(args, "done_uri", None):
150         +     _write_done_marker(
151         +         args.done_uri, video_id, rc, 0, "failed", storage_cfg, scratch
152     @@ -313,24 +366,16 @@ def cmd_infer(args: argparse.Namespace) -> int:
153         )
154
155         # 5. SERIALIZE + 6. PUSH (outputs/<video_id>/?). idempotent on video_id.
156         - out_local = os.path.join(scratch, "outputs", video_id)
157         - os.makedirs(out_local, exist_ok=True)
158         - _write_intervals_csv(segments, os.path.join(out_local, "intervals.csv"))
159         - _write_report_json(
160         + pushed = _serialize_and_push_outputs(
161         +     scratch,
162         +     args.out_uri,
163         +     video_id,
164         +     segments,
165         +     status,
166         -     os.path.join(out_local, "report.json"),
167         -     video_uri=str(getattr(args, "video_uri", "") or ""),
168         +     storage_cfg,

```

```

169 +         str(getattr(args, "video_uri", "") or ""),
170     )
171
172 -     out_prefix = args.out_uri.rstrip("/")
173 -     pushed = []
174 -     for fn in sorted(os.listdir(out_local)):
175 -         uri = f"{out_prefix}/outputs/{video_id}/{fn}"
176 -         storage.put(os.path.join(out_local, fn), uri, config=storage_cfg)
177 -         pushed.append(uri)
178 -
179     print(
180         f"[worker] infer done: {len(segments)} segment(s); pushed {len(pushed)}
artifact(s):"
181     )
182     diff --git a/tests/test_tracknet_hybrid.py b/tests/test_tracknet_hybrid.py
183     index 64909e7..c5ae442 100644
184     --- a/tests/test_tracknet_hybrid.py
185     +++ b/tests/test_tracknet_hybrid.py
186     @@ -5,6 +5,7 @@ healthcheck gating, candidate persistence, AI handoff, and DB
population/linking
187     from unittest.mock import MagicMock, patch
188
189     from backend.pipeline.segmenters.tracknet_hybrid import TrackNetHybridSegmenter
190     +from backend.results import Failure
191
192
193     def _window(start=1.0, end=4.0):
194     @@ -124,7 +125,10 @@ def test_pipeline_healthcheck_failure_aborts(
195
196         seg = TrackNetHybridSegmenter(MagicMock(), {"indexing": {}})
197         res = seg.process_video("v1", "clip.mp4")
198     -     assert res == []
199     +     # A detector-env failure is a Failure, NOT a 0-rally [] ? so the caller marks the
run failed
200     +     # instead of writing an empty "success".
201     +     assert isinstance(res, Failure)
202     +     assert "environment not ready" in res.message
203     +     # Should bail before ever running inference.
204     +     mock_runner_cls.return_value.run_predict.assert_not_called()
205
206     @@ -144,7 +148,36 @@ def test_pipeline_no_trajectory_aborts(
207         mock_provider_registry.get.return_value = MagicMock()
208
209         seg = TrackNetHybridSegmenter(MagicMock(), {"indexing": {}})
210     -     assert seg.process_video("v1", "clip.mp4") == []
211     +     res = seg.process_video("v1", "clip.mp4")
212     +     # A detector timeout/abort (run_predict yielded no trajectory) is a Failure, NOT a
0-rally [] ?
213     +     # this is exactly the wf294 silent-failure that the caller must now mark
failed/retryable.
214     +     assert isinstance(res, Failure)
215     +     assert "no trajectory" in res.message and res.is_recoverable
216     +
217     +
218     +@_patched
219     +def test_pipeline_zero_windows_is_legit_empty_not_failure(
220     +     mock_runner_cls,
221     +     mock_probe,
222     +     mock_dur,
223     +     mock_extract,
224     +     mock_provider_registry,
225     +     mock_env,
226     +):
227     +     """The other side of the wf294 distinction: the detector RAN fine (a trajectory was
produced +

```

```

228 + parsed) but yielded no candidate rally windows. That is a legitimately empty match
? a bare []
229 + (0-rally "success"), NOT a Failure."""
230 + mock_env.return_value = "dummy_key"
231 + mock_dur.return_value = 30.0
232 + mock_extract.return_value = True
233 + mock_runner_cls.return_value = _make_runner(windows=[]) # trajectory ok, zero
windows
234 + mock_provider_registry.get.return_value = MagicMock()
235 +
236 + seg = TrackNetHybridSegmenter(MagicMock(), {"indexing": {}})
237 + res = seg.process_video("v1", "clip.mp4")
238 + assert res == [] and not isinstance(res, Failure)
239 + # The detector DID run (a genuine empty, not an abort).
240 + mock_runner_cls.return_value.run_predict.assert_called_once()
241
242
243 @_patched
244 diff --git a/tests/test_worker.py b/tests/test_worker.py
245 index 96edbee..63cc18e 100644
246 --- a/tests/test_worker.py
247 +++ b/tests/test_worker.py
248 @@ -10,6 +10,7 @@ import json
249     import pytest
250
251     from backend.config import load_config
252 +from backend.results import Failure
253     from backend.worker import __main__ as wmain
254
255     # ----- pure helpers
256 @@ -191,7 +192,7 @@ def test_cmd_infer_local_roundtrip(tmp_path, mocked_pipeline):
257
258
259     class _EmptySeg:
260     -     """A segmenter that finds nothing ? the cold-start failure mode."""
261     +     """A segmenter that finds nothing ? a LEGITIMATE 0-rally match (a bare [])."""
262
263         def __init__(self, db, config):
264             pass
265 @@ -200,6 +201,20 @@ class _EmptySeg:
266             return []
267
268
269     +class _FailSeg:
270     +     """A segmenter whose DETECTOR failed/timed out ? returns a Failure (e.g. WASB
produced no
271     +     trajectory), which is NOT the same as a 0-rally match. This is the wf294 shape."""
272     +
273     +     def __init__(self, db, config):
274     +         pass
275     +
276     +     def process_video(self, video_id, path, max_frames=None):
277     +         return Failure(
278     +             message="WASB detector produced no trajectory (timed out or aborted)",
279     +             is_recoverable=True,
280     +         )
281     +
282     +
283     def test_setup_logging_levels():
284         import logging
285
286 @@ -246,12 +261,56 @@ def test_cmd_infer_zero_segments_is_logged_loudly(tmp_path,
monkeypatch, caplog)
287         assert "0 segments" in msgs and "vidZ" in msgs
288         assert "detector_impl=native" in msgs # surfaces the resolved detector

```



```

289         assert "native runner UNAVAILABLE" in msgs # points at the likely cause
290     - assert (
291     -         json.loads((out / "outputs" / "vidZ" / "report.json").read_text())[
292     -             "segment_count"
293     +         # A segmenter that returns a bare [] is a LEGITIMATE 0-rally match ? status stays
294     +         # (contrast test_cmd_infer_detector_failure_*, where a Failure surfaces as
295     +         # status="failed").
296     +         report = json.loads((out / "outputs" / "vidZ" / "report.json").read_text())
297     +         assert report["segment_count"] == 0 and report["status"] == "success"
298     +
299     + def test_cmd_infer_detector_failure_marks_report_and_marker_failed(
300     +     tmp_path, monkeypatch
301     + ):
302     +     """wf294 fix: a segmenter that returns a Failure (detector timeout/abort) must NOT
303     +     be written
304     +     as a 0-segment "success". cmd_infer returns rc 3 and writes report.json + the done-
305     +     marker with
306     +     status="failed" so the control plane / alpha-user surface can show an error /
307     +     retry."""
308     +     monkeypatch.setattr(wmain, "compute_video_id", lambda p: "vidX")
309     +     monkeypatch.setattr(
310     +         wmain.validator_registry,
311     +         "run_all_with_context",
312     +         lambda path, cfg: ([_OKResult()], {}),
313     +     )
314     +     monkeypatch.setattr(wmain.segmenter_registry, "get", lambda name: _FailSeg)
315     +     monkeypatch.setattr(wmain, "emit_indexer_report", lambda **k: None)
316     +     video = tmp_path / "in.mp4"
317     +     video.write_bytes(b"FAKE")
318     +     out = tmp_path / "bucket"
319     +     done = tmp_path / "bucket" / "_dispatch" / "tok.done"
320     +
321     +     rc = wmain.main(
322     +         [
323     +             "infer",
324     +             "--video-uri",
325     +             str(video),
326     +             "--out-uri",
327     +             str(out),
328     +             "--scratch",
329     +             str(tmp_path / "s"),
330     +             "--segmenter",
331     +             "wasb_hybrid",
332     +             "--done-uri",
333     +             str(done),
334     +         ]
335     +     )
336     +     == 0
337     +
338     + # A detector failure is rc 3 (distinct from validation's rc 2 and a clean rc 0).
339     + assert rc == 3
340     + # report.json is written as an explicit FAILURE, not a false empty "success".
341     + report = json.loads((out / "outputs" / "vidX" / "report.json").read_text())
342     + assert report["status"] == "failed" and report["segment_count"] == 0
343     + assert report["segments"] == []
344     + # ...and the completion marker the cloud dispatcher bucket-polls carries the same
345     + verdict.
346     + marker = json.loads(done.read_text())
347     + assert marker["returncode"] == 3 and marker["status"] == "failed"
348     + assert marker["segment_count"] == 0 and marker["video_id"] == "vidX"
349
350     def test_cmd_infer_validation_failure_returns_2(tmp_path, monkeypatch):

```

5. assistant (2026-07-06T17:29:11.579Z)

I need to verify the claims. Let me check `\_resolve\_provider`, the registration, the default segmenter config, and whether yolo_hybrid imports Failure.

6. user (2026-07-06T17:29:13.476Z)

```
1      """Two-stage rally segmenter: torchvision person detection + ByteTrack tracking
2      (Stage 1, local) feeding an AI provider for semantic boundaries (Stage 2).
3
4      Stage-1 detection now lives in `backend/pipeline/detectors/person_detector.py`
5      (`PersonDetector`, a reusable rally-cue producer ? fusion P1) so the SAME person cue
6      can be consumed by the future fusion segmenter; this segmenter just orchestrates
7      chunking, the AI-provider handoff, and DB population around it. Behaviour is unchanged.
8
9      Stage 1 originally used the AGPL-licensed ultralytics YOLO. To keep the hosted
10     service free of copyleft obligations (see docs/MONETIZATION_AUDIT.md and ADR-005),
11     detection uses torchvision's permissively-licensed (BSD) detectors and the
12     association/tracking that YOLO.track() used to bundle in is provided by ByteTrack
13     via the standalone Apache-2.0 ``trackers`` package (D13/P12 migration, #386 ?
14     previously ``supervision.ByteTrack``, removed upstream in supervision 0.30.0; see
15     ``person_detector.make_tracker`` for the behaviour-pinning details).
16
17     The registry key stays "yolo_hybrid" for back-compat with existing configs and the
18     DB `detector` tags, even though no YOLO is involved anymore.
19     """
20
21     import concurrent.futures
22     import logging
23     import os
24     import time
25     from typing import Any, Dict, List, Tuple
26
27     from backend.pipeline.detectors.person_detector import PersonDetector
28     from backend.pipeline.segmenters.base import BasePlaySegmenter, segmenter_registry
29     from backend.providers import provider_registry
30     from backend.sports import sport_registry
31     from backend.utils.paths import output_base
32     from backend.utils.video import extract_video_chunk, get_video_duration
33
34     logger = logging.getLogger(__name__)
35
36
37     def _fmt_ts(seconds: float) -> str:
38         """Format seconds as HH:MM:SS for cross-verifying a timestamp against the video."""
39         seconds = max(0, int(seconds))
40         h, rem = divmod(seconds, 3600)
41         m, s = divmod(rem, 60)
42         return f"{h:02d}:{m:02d}:{s:02d}"
43
44
45     class HybridYoloSegmenter(BasePlaySegmenter):
46         def __init__(self, db, config):
47             super().__init__(db, config)
48             # Stage-1 person cue (detection + tracking + motion windows) ? a reusable
49             # (fusion P1). This segmenter orchestrates chunking + AI handoff + DB around it.
50             self.person = PersonDetector(config)
51
52             @property
53             def name(self) -> str:
54                 return "yolo_hybrid"
55
56             @property
57             def description(self) -> str:
```

```
58         return (
59             "Two-stage pipeline: torchvision person detection + ByteTrack for fast "
60             "candidate filtering, then an AI provider for high-precision semantic
boundaries."
```

7. user (2026-07-06T17:29:15.051Z)

```
63:     def _resolve_provider(self, video_path: str):
469:         provider = self._resolve_provider(video_path)
```

8. user (2026-07-06T17:29:16.132Z)

```
181:     default_segementer: str = "yolo_hybrid"
275:     @field_validator("default_segementer")
```

9. user (2026-07-06T17:29:17.409Z)

No matches found

10. assistant (2026-07-06T17:29:22.969Z)

Confirmed: yolo_hybrid does NOT import Failure, and `default\_segementer="yolo\_hybrid"`. Let me read `\_resolve\_provider` to confirm the None-return failure paths.

11. user (2026-07-06T17:29:24.982Z)

```
63     def _resolve_provider(self, video_path: str):
64         """Phase 0: resolve sport profile + AI provider (with API key).
65
66         Returns the configured provider instance, or ``None`` to signal that
67         ``process_video`` should early-return ``[]`` (bad sport, missing API key,
68         or unknown provider). Behaviour is identical to the inlined version.
69         """
70         # Get sport profile config
71         sport_name = self.config.sport
72         try:
73             sport_registry.get(sport_name)
74         except KeyError as e:
75             logger.error(str(e))
76             return None
77
78         # Get AI provider and model config
79         provider_name = self.config.ai_provider
80         model_name = self.config.ai_model
81
82         # Get appropriate API key based on the provider
83         api_key_env_var = f"{provider_name.upper()}_API_KEY"
84         api_key = os.environ.get(api_key_env_var)
85         if not api_key:
86             from backend.pipeline.segmenters._keyhint import api_key_hint
87
88             logger.error(
89                 f"{api_key_env_var} environment variable is not set! "
90                 f"To run the {provider_name} segmenter, set your API key. "
91                 + api_key_hint(api_key_env_var)
92             )
93             return None
94
95         try:
96             provider = provider_registry.get(
97                 provider_name, api_key=api_key, model_name=model_name
98             )
99         except KeyError as e:
100             logger.error(str(e))
```

```

101         return None
102
103         logger.info(
104             f"Initializing Hybrid Segmenter (torchvision + ByteTrack) for video:
{video_path}"
105         )
106         return provider
107
108     def _collect_candidate_windows(
109         self,
110         video_id: str,
111         video_path: str,
112         video_duration: float,
113         idx_cfg: Dict[str, Any],
114         chunk_duration: float,
115         chunk_overlap: float,
116         velocity_thresh: float,
117         merge_gap: float,
118         frame_skip: int,
119         chunks_dir: str,
120     ) -> List[Dict[str, Any]]:
121         """Phase 2: chunk the video, extract per-chunk action windows (pipelined or
122         sequential), then dedup-merge overlapping windows across chunks. Verbatim
123         move."""
124
125         step = chunk_duration - chunk_overlap
126         chunk_starts = []
127         t = 0.0
128         while t < video_duration:
129             chunk_starts.append(t)
130             t += step
131
132         num_chunks = len(chunk_starts)
133         logger.info(
134             f"Video duration: {video_duration:.1f}s. Splitting into {num_chunks}
135             chunks."
136         )
137
138         all_candidate_windows = []
139
140         t_start = time.time()
141         prefetch_workers = idx_cfg.yolo_prefetch_workers
142
143         if num_chunks > 1 and prefetch_workers > 0:
144             # --- PIPELINED PROCESSING ---
145             # GPU inference must stay sequential (single GPU), but ffmpeg chunk
146             # extraction is pure I/O. We pre-extract upcoming chunks in background
147             # threads so extraction of chunk N+1 overlaps with inference on chunk N.
148             logger.info(
149                 f"Running detection Phase 2 with pipelined extraction
150                 ({prefetch_workers} prefetch workers)..."
151             )
152
153             extract_executor = concurrent.futures.ThreadPoolExecutor(
154                 max_workers=prefetch_workers
155             )
156
157             def _extract_chunk(ci: int, cs: float) -> Tuple[int, float, str, bool]:
158                 chunk_end = min(cs + chunk_duration, video_duration)
159                 actual_dur = chunk_end - cs
160                 chunk_path = os.path.join(chunks_dir, f"chunk_{video_id}_{ci}.mp4")
161                 success = extract_video_chunk(video_path, cs, actual_dur, chunk_path)
162                 return (ci, cs, chunk_path, success)
163
164             # Submit all extractions upfront ? they'll queue and run as workers free up
165             extract_futures = [

```

```

162         extract_executor.submit(_extract_chunk, ci, cs)
163         for ci, cs in enumerate(chunk_starts)
164     ]
165
166     # Process results in order: wait for extraction, then run inference on GPU
167     for future in extract_futures:
168         ci, chunk_start, chunk_path, success = future.result()
169         chunk_end = min(chunk_start + chunk_duration, video_duration)
170         logger.info(
171             f"Processing Chunk {ci + 1}/{num_chunks} ({chunk_start:.1f}s -
{chunk_end:.1f}s)..."
172         )
173
174         if not success:
175             logger.warning(f"Chunk {ci + 1} extraction failed. Skipping.")
176             continue
177
178         windows = self.person.extract_action_windows_from_chunk(
179             chunk_path,
180             chunk_start,
181             velocity_thresh,
182             merge_gap,
183             frame_skip=frame_skip,
184             chunk_index=ci,
185         )
186         logger.info(
187             f"Found {len(windows)} candidate action windows in chunk {ci + 1}."
188         )
189         all_candidate_windows.extend(windows)
190
191         try:
192             os.remove(chunk_path)

```

12. user (2026-07-06T17:29:25.601Z)

```

559     # Register the segmenter
560     segmenter_registry.register(HybridYoloSegmenter)
561

```

13. assistant (2026-07-06T17:29:33.598Z)

All claims verified. Now let me confirm the cmd_infer path treats bare `[ ]` as success (status="success", rc=0) ? i.e., that the wf294 shape genuinely reproduces for yolo_hybrid.

14. user (2026-07-06T17:29:35.046Z)

```

300         return rc
301
302     segments: List[dict] = []
303     segmenter_actual = None
304     segmentation_ran = False
305     status = "failed"
306     try:
307         if failures:
308             print(
309                 "[worker] validation FAILED: "
310                 + "; ".join(f"{r.validator_name}: {r.message}" for r in failures)
311             )
312             return _fail(2)
313     seg_name = args.segmenter or config.indexing.default_segmenter
314     segmenter_cls = segmenter_registry.get(seg_name)
315     if not segmenter_cls:
316         print(
317             f"[worker] unknown segmenter: {seg_name!r} "

```

```

318         f"(have {list(segmenter_registry.list_available())})"
319     )
320     return _fail(2)
321     segmenter_actual = seg_name
322     result = segmenter_cls(db, config).process_video(
323         video_id, local_video, max_frames=args.max_frames
324     )
325     segmentation_ran = True
326     if isinstance(result, Failure):
327         print(f"[worker] segmenter FAILED: {result.message}")
328         return _fail(3)
329     segments = result.value if hasattr(result, "value") else result
330     status = "success"
331     # Loud, never-silent diagnostic: a 0-segment success is almost always a
misconfig, not an
332     # empty match ? the cold-start failure mode (the substrate detector yielded no
usable
333     # trajectories). Explain it so a run is analysable from the log alone (re-run -v
for more).
334     if not segments:
335         detector_impl = (
336             os.environ.get("RALLY_DETECTOR_IMPL")
337             or getattr(config.indexing, "detector_impl", None)
338             or "auto"
339         )
340         logger.warning(
341             "infer produced 0 segments for video_id=%s (segmenter=%s,
detector_impl=%s). "
342             "The detector substrate yielded no usable trajectories/windows ? check
the detector "
343             "logs above; common causes: a missing ${AI_PROVIDER}_API_KEY with "
344             "skip_ai_handoff=False (the segmenter bails BEFORE detection ? set the
key or "
345             "indexing.skip_ai_handoff=true; cloud-serving now fails this at
startup), native "
346             "runner UNAVAILABLE (weights/repo/WASB_PYTHON env), the WASB env
produced no "
347             "detections, or the input resolution differs from the tuned proxy
resolution. "
348             "Re-run with -v for the native command + frame counts.",
349             video_id,
350             segmenter_actual,
351             detector_impl,
352         )
353     finally:
354         # Best-effort per-video observability report (never raises) ? parity with the
main CLI.
355         emit_indexer_report(
356             db=db,
357             video_id=video_id,
358             video_path=local_video,
359             config=config,
360             val_results=val_results,
361             val_context=val_context,
362             segmenter_requested=args.segmenter,
363             segmenter_actual=segmenter_actual,
364             was_reingest=False,
365             segmentation_ran=segmentation_ran,
366         )
367
368     # 5. SERIALIZE + 6. PUSH (outputs/<video_id>/?). idempotent on video_id.
369     pushed = _serialize_and_push_outputs(

```

This fully confirms the finding. The `cmd_infer` path at line 326 checks `isinstance(result, Failure)` ? rc 3`. But when `yolo_hybrid` returns a bare `[]`` (its only failure representation), it falls through to line 329-330: `segments = []`, `status = "success"`. This is exactly the wf294 silent-failure shape.`

The finding is accurate on every load-bearing claim:

- `yolo_hybrid` does NOT import `Failure` (grep found no matches).
- `_resolve_provider` returns `None` on missing key (line 93), unknown sport (line 76), unknown provider (line 101).`
- `process_video` returns bare `[]` at line 471 (provider None) and line 482 (video_duration<=0).`
- Return annotation is `List[Dict[str, Any]]` (line 465), no Failure branch.`
- Registered (line 560), `default_segmenter="yolo_hybrid"` (models.py:181).`
- The wf294 diff touched only `trajectory_hybrid.py + worker/__main__ + tests; yolo_hybrid.py` untouched.
- `cmd_infer's generic `isinstance(result, Failure)` branch (line 326) can never fire for yolo_hybrid, so bare `[]` ? status="success"/0 segments (lines 329-330).`

The only nuance for severity: the finding's own scenario (default segmenter `yolo_hybrid` + missing `GEMINI_API_KEY`) ? note default `ai_provider` would need checking, but the failure exists regardless of which provider (missing `{PROVIDER}_API_KEY`).` This is a real, reproducible silent-failure hole in the default segmenter, left unaddressed by the twin fix. This is a genuine correctness/parity inconsistency, matching "high" severity as a same-class hole in the default code path.

16. user (2026-07-06T17:29:52.707Z)

Structured output provided successfully